Package 'rawData'

April 18, 2026

Type Package

Title RAW data manager

Version 0.2.14

Maintainer Thomas Gredig <thomas.gredig@csulb.edu>

Description Store all scientific RAW data in one repository for analysis. This RAW data manager keeps track of files by assigning a permanent ID. The data files are tracked by a cyclic redundancy code to keep the unique file ID. If files are renamed, moved, or copied, the ID is preserved. The same files can be managed in different file systems, as the RAW Data manager checks for best availability of the data. Data from common instruments is imported into data frames that hold all data grouped by ID.

License GPL (>= 3)

Encoding UTF-8

LazyData true

RoxygenNote 7.3.3

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

Imports DBI,
RSQLite,
cli,
desc,
dplyr,
here,
methods,
nanoAFMr (>= 2.3),
rigakuXRD (>= 0.4.1),
stats,
tools,
usethis,
utils

Depends R (>= 4.0)

Contents

`.cleanRawBase` *Cleans the rawBase Variables*

Description

Any variable that contains "Paths" will have duplicate paths removed.

Usage

```
.cleanRawBase(rawBase)
```

Arguments

rawBase	S3 object

Value

updated rawBase object

Examples

```
rawBase = list(varWithDuplicates = c("letters","A","B","A","A","B","C"),
               myPaths = c("letters","A","B","A","A","B","C"))
.cleanRawBase(rawBase)
```

.getFileType *Returns File Type*

Description

Returns File Type

Usage

```
.getFileType(filename)
```

.getSQLtableNames *Returns SQLite table names*

Description

Returns SQLite table names

Usage

```
.getSQLtableNames()
```

.readSQLhistory *reads the sqlHistory table*

Description

reads the sqlHistory table

Usage

```
.readSQLhistory(mydb)
```

Arguments

mydb	database connection from DBI::dbConnect

.updateSQLhistory *updates the sqlHistory table*

Description

updates the sqlHistory table

Usage

```
.updateSQLhistory(mydb, token, description)
```

Arguments

mydb	database connection from DBI::dbConnect
token	a number representing the time
description	string with description of update

.writeSQLdatabaseInit *writes a SQLite database initialization*

Description

writes a SQLite database initialization

Usage

```
.writeSQLdatabaseInit(mydb, verbose = FALSE)
```

Arguments

mydb	database connection from DBI::dbConnect
verbose	logical to output extra information

create_dataRAW *Constructor of dataRAW S3 class*

Description

Constructor of dataRAW S3 class

Usage

```
create_dataRAW(
  ID,
  raw_paths,
  filename,
  crc = NULL,
  size = NULL,
  type = NULL,
  missing = NULL,
  altered = NULL,
  sample = NULL,
  date = NULL,
  meta = NULL
)
```

Arguments

ID	unique ID for file
raw_paths	vector with paths to search files
filename	filename including path
crc	128-bit MD5 unique hash
size	file size in bytes
type	data type of file
missing	logical, file cannot be found
altered	logical, file likely been altered
sample	string of sample name
date	date for data recording
meta	additional data in JSON format (use jsonlite)

---

create_rawBase *Constructor of rawBase S3 class*

Description

Constructor of rawBase S3 class

Usage

```
create_rawBase(
  projectName,
  pkgName = NULL,
  paths = NULL,
  recursive = TRUE,
  sqlPaths = NULL,
  instrument_list = NULL,
  legacyRAWIDfile = NULL,
  extensions = c()
)
```

Arguments

projectName	project name
pkgName	name of the package
paths	path or paths with data files
recursive	logical weather to search paths recursively
sqlPaths	paths for location of SQL database
instrument_list	vector with instruments to be updated
legacyRAWIDfile	full path and file name of RAW-ID.csv file
ext	any extensions as a list

---

get_test_RAW_folder *Create a Test RAW folder with text data files*

Description

Create a Test RAW folder with text data files

Usage

```
get_test_RAW_folder(n, projectName, recreate = FALSE)
```

Arguments

n	number of files
projectName	string for name of the project
returns	folder with temporary files

Examples

```
dir(get_test_RAW_folder(2,"spinPc"))
```

---

instrumentAFM *Load AFM instrument data*

Description

Load AFM instrument data

Usage

```
instrumentAFM(rawBase)
```

Arguments

rawBase	rawBase object

Value

data frame with AFM data for all AFM files in rawBase

instrumentXRD *Load XRD instrument data*

Description

Load XRD instrument data

Usage

```
instrumentXRD(rawBase)
```

Arguments

rawBase	rawBase object

Value

rawBase

print.dataRAW *Print method for the dataRAW class*

Description

Print method for the dataRAW class

Usage

```
## S3 method for class 'dataRAW'
print(d, row.names = FALSE, ...)
```

Arguments

row.names	logical to output additional row names
...	additional params
RAW	created with create_dataRAW() function

print.rawBase *print rawData base object*

Description

print rawData base object

Usage

```
## S3 method for class 'rawBase'
print(rawBase, ...)
```

raw.addFiles *Add dataRAW with new files*

Description

Finds files that match the project name and are located in the path. Then appends those files to the dataRAW; if the file already exists it will be updated, but not added; the ID remains the same.

Usage

```
raw.addFiles(rawBase, verbose = FALSE)
```

Arguments

rawBase	see raw.init() to create this
verbose	logical to output more information

Value

rawBase object

raw.addInstrument *Add Instruments*

Description

Add Instruments

Usage

```
raw.addInstrument(rawBase, instrument_list)
```

Arguments

rawBase	rawBase object
intrument_list	list with instrument name (XRD, AFM) and function to call

See Also

[raw.init()]

`raw.addPath` *Add raw-data and SQL paths to a rawBase object*

Description

Adds a new raw-data path to ‘rawBase$raw_paths‘ and records the operation in the import history. Optionally, a SQL path can also be added to ‘rawBase$sql_paths‘ if it exists on disk. After updating paths and history, the rawBase object is cleaned to remove duplicate entries and apply any internal normalization performed by [.cleanRawBase()].

Usage

```
raw.addPath(rawBase, project, path = NULL, sqlPath = NULL, recursive = TRUE)
```

Arguments

`rawBase`	A rawBase object.
`project`	A character string identifying the project associated with the path addition. This value is recorded in the import history.
`path`	A character string giving the raw-data directory to add. If ‘NULL‘, no raw-data path is appended.
`sqlPath`	A character string giving the SQL directory to add. If ‘NULL‘, no SQL path is considered. The path is only added if ‘dir.exists(sqlPath)‘ returns ‘TRUE‘.
`recursive`	Logical; passed to [update_rawBaseHistory()] to indicate whether the path addition should be treated as recursive. Defaults to ‘TRUE‘.

Details

The function performs three updates:

1. Appends ‘path‘ to ‘rawBase$raw_paths‘ when ‘path‘ is not ‘NULL‘.
2. Records the action in ‘rawBase$import_history‘ using [update_rawBaseHistory()] with action ‘"add"‘.
3. Appends ‘sqlPath‘ to ‘rawBase$sql_paths‘ when ‘sqlPath‘ is not ‘NULL‘ and the directory exists.

Finally, the updated object is passed to [.cleanRawBase()] to remove duplicate paths and perform any additional cleanup.

Value

An updated rawBase object.

See Also

[update_rawBaseHistory()], [.cleanRawBase()]

raw.addSQLpath *Add an SQL path to a rawBase object*

Description

Adds a directory to ‘rawBase$sql_paths‘ if the supplied path is not ‘NULL‘ and exists on disk. Duplicate entries are removed after the new path is added.

Usage

```
raw.addSQLpath(rawBase, sqlPath)
```

Arguments

rawBase	A rawBase object.
sqlPath	A character string giving the SQL directory to add. If ‘NULL‘, the object is returned unchanged. The path is only added if ‘dir.exists(sqlPath)‘ returns ‘TRUE‘.

Value

An updated rawBase object with ‘sql_paths‘ modified and deduplicated.

raw.checkMissing *Check missing files*

Description

Check missing files

Usage

```
raw.checkMissing(rawBase)
```

Arguments

dataRAW	dataRAW S3 object with files

Value

dataRAW object with updated missing fields

raw.find *Find project files*

Description

Uses the rawBase list to include instructions on how to search files. Searches files that contain an 8 digit date followed by underscore or dash and includes the project name, something like 20240822_something_ProjName_something

If no path is provided, then it will search in the current path or if in interactive mode, will prompt for a path.

Usage

```
raw.find(rawBase, recursive = TRUE, quiet = FALSE)
```

Arguments

rawBase	list generated with raw.init()
recursive	logical, determines whether path is searched recursively.
quiet	suppress messages

Value

vector with file list that includes paths

raw.getCRC *Returns the CRC for filename*

Description

Returns the CRC for filename

Usage

```
raw.getCRC(filename)
```

Arguments

filename	filename

raw.getDatabase *Manage SQL Database with Data Package*

Description

Some data packages require large amounts of data (1 - 10GB), which cannot be stored in the 'extdata' folder directly, since the install would take too long. If data exceeds about 10 MB, then RDA files become inefficient, and an SQL database makes sense. For AFM data, you can use AFM.writeDB() for example.

The database needs to be stored in the 'extdata' and also needs to be version controlled. This function helps manage this process. The data package generates the small RDA data files and puts the large data files into the SQL databse that is stored in the main directory of the database. The database needs to store at least one more dummy file in the 'inst/extdata' folder, so that this folder is generated and loaded.

When called with the 'pkgname', the function uses the version to generate the database filename and return its path.

Usage

```
raw.getDatabase(rawBase, verbose = FALSE)
```

Arguments

rawBase	rawBase object
verbose	logical, additional information

Value

SQL database filename and path or NULL or empty string if not found

raw.getFilename *Full Path and Filename for dataRAW object*

Description

Full Path and Filename for dataRAW object

Usage

```
raw.getFilename(rawBase, ID)
```

Arguments

ID	dataRAW object ID
dataRAW	dataRAW object

Value

full path of filename, if not founds returns empty NA

raw.getMD5 *returns the MD5sum of a filename*

Description

returns the MD5sum of a filename

Usage

```
raw.getMD5(filename, num = 6)
```

Arguments

filename	filename
num	string length

Value

CRC MD5 sum for this particular file

Examples

```
raw.getMD5(raw.getSampleFiles(),num=10)
```

raw.getMD5str *returns a string with all MD5 files*

Description

returns a string with all MD5 files

Usage

```
raw.getMD5str(filename)
```

Arguments

filename	filename with path

Value

CRC MD5 sum for this particular file

Examples

```
raw.getMD5str(raw.getSampleFiles())
```

raw.getPartialMD5 *returns the MD5sum of a filename*

Description

returns the MD5sum of a filename

Usage

```
raw.getPartialMD5(filename, num = 6)
```

Arguments

filename	filename
num	string length

Value

CRC MD5 sum for this particular file

raw.getPartialMD5str *returns a string with all MD5 files*

Description

returns a string with all MD5 files

Usage

```
raw.getPartialMD5str(filename)
```

Arguments

filename	filename with path

Value

CRC MD5 sum for this particular file

raw.getSampleFiles *provide sample RAW data files*

Description

provide sample RAW data files

Usage

```
raw.getSampleFiles()
```

Value

path and filename with sample data

raw.getType *Type of rawData item*

Description

Type of rawData item

Usage

```
raw.getType(rawData, ID)
```

Value

string with type, such as "XRD", "AFM", etc.

raw.importRAWID *Imports legacy RAW-ID.csv*

Description

Imports legacy RAW-ID.csv

Usage

```
raw.importRAWID(rawBase)
```

Arguments

rawBase	object that contains information with the legacy coded

Value

updated rawBase object

raw.init *rawBase initialization*

Description

the rawBase list contains key information about how data is stored in the dataProject; this includes folder information, etc. - this function is called to initialize this list.

Will prompt for path to search for data files of that project and also a path to store the SQL data.

Usage

```
raw.init(
  projectName,
  paths = NULL,
  sqlPaths = NULL,
  recursive = TRUE,
  instrument_list = list(XRD = instrumentXRD, AFM = instrumentAFM),
  legacyRAWIDfile = "",
  extensions = c(),
  ...
)
```

Arguments

projectName	short string with the project name "spinPc"
paths	path or paths with data files
sqlPaths	paths for location of SQL database
recursive	logical weather to search paths resursively
instrument_list	list with instruments to be updated
legacyRAWIDfile	filename with RAW-ID.csv data file (legacy code)
extensions	list with file extensions (does not require project name)
...	parameters for raw.addFiles, such as verbose

Value

rawBase object

raw.initDB *Initialize SQL database*

Description

Create and initialize a database, if none exists. The SQLite database can store images and large data files, which would otherwise slow down the R package. The database contains the name of the package, includes the version number and ends with .sqlite. Before creating a new database, it checks whether a database already exists; it might have an older version. If so, then it updates (renames) the file to correspond to the latest version. The rawBase object stores information to what is stored in this separate database, in case it gets lost or needs to be recreated.

Usage

```
raw.initDB(rawBase, quiet = FALSE)
```

Arguments

rawBase	object, use create_rawBase()
quiet	suppress messages

raw.readRAWIDheader *Reads an RAW ID header*

Description

Header information is presided by # and separated name:value with colon for example # Version: 1.0

Usage

```
raw.readRAWIDheader(fIDfile)
```

Arguments

fIDfile	path and file name for RAW-ID.csv file

raw.showHistoryDB *Prints the history of the SQLite database*

Description

Prints the history of the SQLite database

Usage

```
raw.showHistoryDB(rawBase)
```

raw.update *Update dataRAW with new files*

Description

Finds files that match the project name and are located in the path. Then appends those files to the dataRAW.

Usage

```
raw.update(
  rawBase,
  path = NULL,
  project = NULL,
  file_pattern = NULL,
  sqlPath = NULL,
  recursive = TRUE,
  instrument_list = list(XRD = instrumentXRD, AFM = instrumentAFM),
  extensions = character(),
  forceUpdateMissing = FALSE,
  quiet = FALSE
)
```

Arguments

rawBase	object with information about file locations
path	path with updated information about files.
project	any new project to be added to be searched
file_pattern	pattern to search for additional files to be included
sqlPath	any SQL path to be added
recursive	logical, recursive search in path
instrument_list	list, contains instruments to be added
forceUpdateMissing	if TRUE, all missing files are checked for folder update, useful for legacy RAWID files
quiet	suppresses messages

Value

rawBase object

raw.updateDB *Update the SQLite database or re-generate it*

Description

Update the SQLite database or re-generate it

Usage

```
raw.updateDB(rawBase, quiet = FALSE)
```

raw.updateInstrument *Updates data from instruments into package*

Description

The instrument functions are added to rawBase when it is created with create_rawBase() for example

Usage

```
raw.updateInstrument(rawBase, quiet = FALSE)
```

Arguments

rawBase	object with information
quiet	suppresses messages

raw.updateMissingFilePaths *Update paths for missing raw files*

Description

Searches for files in ‘rawBase$dataRAW‘ that are currently marked as missing and attempts to locate them under the directories listed in ‘rawBase$raw_paths‘. Matching is based on filename and CRC, assuming the filename itself has not changed. When a match is found, the corresponding ‘path‘ entry is updated. After all candidate files are processed, missing-file status is recomputed with [raw.checkMissing()].

Usage

```
raw.updateMissingFilePaths(rawBase)
```

Arguments

rawBase	A raw-data database object containing at least:
	'dataRAW' A data frame-like object with columns 'ID', 'filename', 'crc', 'path', and 'missing'.
	'raw_paths' A character vector of base directories to search for relocated files.

Details

If no files are marked as missing, the function leaves 'rawBase' unchanged and emits an informational message.

For each row in 'rawBase$dataRAW' with 'missing == TRUE', the function calls '.findSubPath()' using the stored filename and CRC. If a matching file is found, the file path is updated in 'dataRAW'. Once all missing files have been checked, the modified table is written back into 'rawBase', and [raw.checkMissing()] is called to refresh the 'missing' flags.

The function assumes that missing files may have been moved to a different directory, but that their filenames remain unchanged.

Value

The updated 'rawBase' object, with revised file paths where matches were found and refreshed missing-file status.

See Also

[raw.checkMissing()], '.findSubPath()'

Examples

```
## Not run:
rawBase <- raw.updateMissingFilePaths(rawBase)

## End(Not run)
```

---

rbind.dataRAW *row bind two dataRAW sets*

Description

row bind two dataRAW sets

Usage

```
## S3 method for class 'dataRAW'
rbind(d1, d2)
```

Arguments

d1	first dataRAW object
d2	second dataRAW object to be appended

Index